GUÍA DE LABORATORIO

(formato docente)

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Pantallas múltiples con NavigationBar y comunicación entre Composables en Jetpack Compose				
NÚMERO DE PRÁCTICA:	5	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		2
FECHA INICIO:	29/09/2026	**FECHA FIN:**	03/10/2026	**DURACIÓN:**	100 minutos
RECURSOS A UTILIZAR: Android Studio (Quail o superior), Kotlin, Jetpack Compose, librería Navigation Compose, emulador Android o dispositivo físico (API 34 o superior)					
DOCENTE(s): Roxana Evelyn Limache Calatayud					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender la estructura de una aplicación con múltiples pantallas organizadas mediante una barra de navegación inferior (NavigationBar) en Jetpack Compose. - Implementar una aplicación con tres pantallas (Home, Edificios y Mapa) navegables desde una barra de navegación inferior. - Comparar dos formas de comunicar datos entre Composables: mediante funciones lambda y mediante un ViewModel compartido.	
TEMAS: - NavigationBar y NavHost en Jetpack Compose - Comunicación entre Composables mediante funciones lambda - ViewModel compartido (Shared ViewModel) entre pantallas	
COMPETENCIAS	C.a Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas de ingeniería necesarias para la construcción de software e implementación de sistemas de información.
	C.b Investiga nuevos modelos, metodologías, técnicas, herramientas y tecnologías por ser necesarias para mantener la vigencia en el desempeño profesional.
	C.c
	C.d

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

Revisar los siguientes recursos antes de la sesión:

- Android Developers, "Navigation bar | Jetpack Compose":
https://developer.android.com/develop/ui/compose/components/navigation-bar

- Android Developers, navegación con Compose:
https://developer.android.com/develop/ui/compose/navigation

¿Por qué organizar una app en varias pantallas con navegación inferior?

En el paradigma clásico de Views, una aplicación con varias secciones (por ejemplo Home, una lista y un mapa) se organizaba con una sola Activity que alojaba distintos Fragments, intercambiados dentro de un FragmentContainerView y sincronizados con una BottomNavigationView. En Jetpack Compose el mismo problema se resuelve sin Fragments: cada "sección" es una función @Composable independiente, y es Navigation Compose el que decide cuál mostrar, coordinado con una barra de navegación inferior (NavigationBar) construida con Material3.

NavigationBar y NavHost en Jetpack Compose

Un Scaffold puede recibir un parámetro bottomBar con una NavigationBar que contiene un NavigationBarItem por cada sección (con su ícono, etiqueta y estado seleccionado). Cada NavigationBarItem, al ser pulsado, invoca navController.navigate(ruta), donde navController es una instancia de NavController creada con rememberNavController(). El contenido central del Scaffold aloja un NavHost que declara, mediante composable(route) { ... }, qué función @Composable se muestra para cada ruta. El estado de selección de la barra se sincroniza leyendo la ruta actual con currentBackStackEntryAsState(), como se resume en la Figura 1.

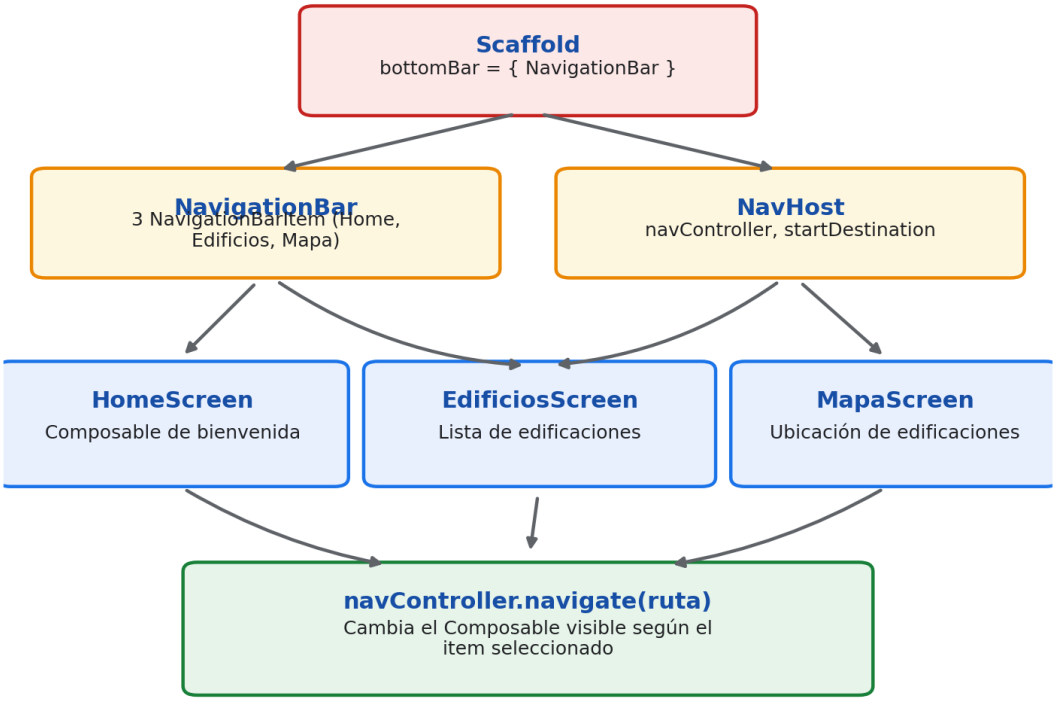


Figura 1. Estructura de una app con Scaffold, NavigationBar y NavHost en Jetpack Compose (elaboración propia).

Comunicación entre Composables: lambda vs. ViewModel compartido

Cuando una pantalla necesita enterarse de algo que ocurrió en otra (por ejemplo, que se seleccionó un elemento en una lista), existen dos formas habituales de comunicarlas en Compose. La primera es mediante una función lambda: el Composable que origina el evento recibe un parámetro de tipo función (por ejemplo onItemSeleccionado: (String) -> Unit) y lo invoca cuando ocurre el evento; quien lo instancia (usualmente el NavHost o un Composable padre) decide qué hacer con ese dato. La segunda es mediante un ViewModel compartido: ambas pantallas obtienen la misma instancia de un ViewModel (con el mismo scope, por ejemplo el del NavGraph), una escribe el valor y la otra lo observa con collectAsState(). La Figura 2 compara ambos flujos; a diferencia de la comunicación entre Fragments mediante interfaces o el FragmentManager, en Compose ninguna de las dos opciones requiere una API adicional de ciclo de vida.

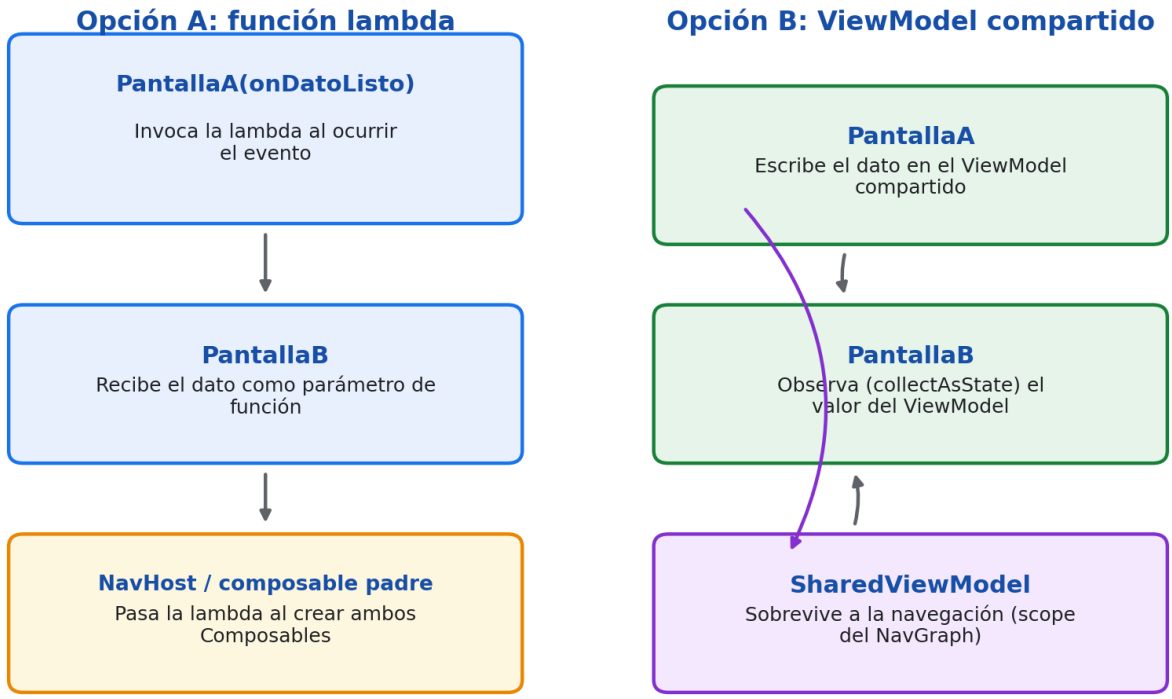


Figura 2. Comparación entre comunicar Composables mediante función lambda y mediante un ViewModel compartido (elaboración propia).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para construir una aplicación con tres pantallas (Home, Edificios y Mapa) navegables desde una barra inferior, comunicando la selección de un edificio mediante una función lambda. También puede apoyarse en el siguiente recurso:

Referencia: "Navigation bar | Jetpack Compose" — https://developer.android.com/develop/ui/compose/components/navigation-bar

Paso 1. Crear un nuevo proyecto y definir una sealed class Screen con tres objetos: Home, Edificios y Mapa, cada uno con su propiedad route (String) y label (String).

nombre sugerido para el proyecto: "CampusNavigator_Compose", Package name: com.example.campusnavigator_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la

plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

Código:

```
sealed class Screen(val route: String, val label: String) {
    object Home : Screen("home", "Home")
    object Edificios : Screen("edificios", "Edificios")
    object Mapa : Screen("mapa", "Mapa")
}
```

Importaciones que se usarán en los siguientes pasos (agregar junto a las demás importaciones del archivo):

```
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Home
import androidx.compose.material.icons.filled.List
import androidx.compose.material.icons.filled.Place
import androidx.compose.material3.Button
import androidx.compose.material3.Icon
import androidx.compose.material3.NavigationBar
import androidx.compose.material3.NavigationBarItem
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import androidx.navigation.compose.NavHost
import androidx.navigation.compose.composable
import androidx.navigation.compose.currentBackStackEntryAsState
import androidx.navigation.compose.rememberNavController
```

Paso 2. Verificar que el proyecto tenga las dependencias necesarias: abrir build.gradle.kts (Module :app) y confirmar que el bloque dependencies { ... } incluya implementation("androidx.navigation:navigation-compose:2.8.0") (para NavHost/NavController) e implementation("androidx.compose.material:material-icons-extended") (para los íconos Icons.Default.Home/List/Place de la barra de navegación); si falta alguna, agregarla y sincronizar el proyecto (botón “Sync Now”) antes de continuar.

Paso 3. En el Composable raíz, crear val navController = rememberNavController() y una variable de estado var edificioSeleccionado by remember { mutableStateOf("Ninguno") }.

Código (Composable raíz):

```
@Composable
fun MainScreen() {
    val navController = rememberNavController()
    var edificioSeleccionado by remember { mutableStateOf("Ninguno") }
```

```
    // Contenido en los siguientes pasos
}
```

Paso 4. Envolver la UI en un Scaffold(bottomBar = { ... }). Dentro de bottomBar, declarar una NavigationBar con un NavigationBarItem por cada Screen, marcando como seleccionado el que coincide con la ruta actual (currentBackStackEntryAsState()) y llamando a navController.navigate(screen.route) al pulsarlo.

Código (dentro de MainScreen):

```
val items = listOf(Screen.Home, Screen.Edificios, Screen.Mapa)

Scaffold(
    bottomBar = {
        NavigationBar {
            val navBackStackEntry by navController.currentBackStackEntryAsState()
            val currentRoute = navBackStackEntry?.destination?.route

            items.forEach { screen ->
                val icon = when (screen) {
                    Screen.Home -> Icons.Default.Home
                    Screen.Edificios -> Icons.Default.List
                    Screen.Mapa -> Icons.Default.Place
                }
                NavigationBarItem(
                    selected = currentRoute == screen.route,
                    onClick = { navController.navigate(screen.route) },
                    icon = { Icon(icon, contentDescription = screen.label) },
                    label = { Text(screen.label) }
                )
            }
        }
    }
) { padding ->
    // Contenido del NavHost en el siguiente paso
}
```

Paso 5. Dentro del padding entregado por el Scaffold, declarar un NavHost(navController, startDestination = Screen.Home.route) con un bloque composable(route) { ... } para cada una de las tres pantallas.

Código (dentro del Scaffold, en el content lambda):

```
NavHost(
    navController = navController,
    startDestination = Screen.Home.route,
    modifier = Modifier.padding(padding)
) {
    composable(Screen.Home.route) {
        HomeScreen(edificioSeleccionado = edificioSeleccionado)
    }
    composable(Screen.Edificios.route) {
        EdificiosScreen(
            onEdificioSeleccionado = { nombre ->
                edificioSeleccionado = nombre
            }
        )
    }
    composable(Screen.Mapa.route) {
        MapaScreen()
    }
}
```

Paso 6. Implementar HomeScreen(edificioSeleccionado: String) mostrando un Text de bienvenida y otro Text con el valor recibido (por ejemplo "Último edificio consultado: $edificioSeleccionado").

Código:

```
@Composable
fun HomeScreen(edificioSeleccionado: String) {
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp),
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Text(text = "Bienvenido a la app de Edificios")
        Text(text = "Último edificio consultado: $edificioSeleccionado")
    }
}
```

Paso 7. Implementar EdificiosScreen(onEdificioSeleccionado: (String) -> Unit) con una LazyColumn que liste al menos 4 edificaciones (nombre + botón); al pulsar un ítem, invocar onEdificioSeleccionado(nombre).

Código:

```
@Composable
fun EdificiosScreen(onEdificioSeleccionado: (String) -> Unit) {
    val edificios = listOf("Biblioteca Central", "Pabellón A", "Pabellón B", "Auditorio")

    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {
        items(edificios) { nombre ->
            Row(
                modifier = Modifier
                    .fillMaxWidth()
                    .padding(vertical = 8.dp),
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text(text = nombre, modifier = Modifier.weight(1f))
                Button(onClick = { onEdificioSeleccionado(nombre) }) {
                    Text("Ver")
                }
            }
        }
    }
}
```

Paso 8. Implementar MapaScreen() con un Text de marcador de posición (por ejemplo "Mapa de ubicaciones (pendiente de integrar)"), y al declarar EdificiosScreen en el NavHost, pasarle una lambda que actualice edificioSeleccionado en el Composable raíz, como se resume en la Figura 2.

Código:

```
@Composable
fun MapaScreen() {
    Box(
        modifier = Modifier.fillMaxSize(),
        contentAlignment = Alignment.Center
```

```
    ) {
        Text(text = "Mapa de ubicaciones (pendiente de integrar)")
    }
}
```

La lambda que actualiza edificioSeleccionado ya se declaró en el Paso 5, al definir composable(Screen.Edificios.route) dentro del NavHost.

Paso 9. En MainActivity, dentro del método onCreate(), reemplazar el contenido de setContent { } (el que trae la plantilla por defecto, con Scaffold y Greeting) para que invoque MainScreen() en su lugar.

Código (dentro de onCreate):

```
setContent {
    CampusNavigator_ComposeTheme {
        MainScreen()
    }
}
```

Paso 10. Ejecutar la aplicación en un emulador y verificar que, al pulsar cada ítem de la barra inferior, se muestra la pantalla correspondiente conservando el resaltado del ítem activo, siguiendo la estructura de la Figura 1; luego, seleccionar un edificio en Edificios y confirmar que HomeScreen muestra el valor actualizado al volver a esa pestaña.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Tomando como referencia el ejercicio resuelto por el docente, reemplaza la comunicación por lambda entre Edificios y Home por un ViewModel compartido, y agrega una cuarta pestaña:

1. Crea una clase SeleccionViewModel : ViewModel() con una propiedad mutableStateOf<String> (o StateFlow) para almacenar el edificio seleccionado.
2. Obtén una única instancia de SeleccionViewModel con viewModel() en el Composable raíz y pásala a EdificiosScreen y a HomeScreen (en lugar de la lambda del ejercicio resuelto).
3. En EdificiosScreen, al seleccionar un edificio, actualiza directamente el valor en el ViewModel en vez de invocar una función lambda.
4. En HomeScreen, lee el valor del ViewModel (con collectAsState() si usaste StateFlow) y muéstralo, verificando que el comportamiento visual sea el mismo que con la lambda.
5. Agrega una cuarta pestaña "Perfil" (u otra que prefieras) con un Composable simple de marcador de posición, y su NavigationBarItem correspondiente.

Entregable: informe con capturas de pantalla de las cuatro pestañas funcionando y de la comunicación mediante ViewModel compartido, y una breve descripción de la implementación.

IV. CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (pantallas Home, Edificios y Mapa navegables mediante NavigationBar, donde el edificio seleccionado en Edificios se refleja en Home)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.
2. En tu propia implementación, ¿qué diferencia encontraste entre comunicar Composables mediante una función lambda y hacerlo mediante un ViewModel compartido? ¿Cuál de las dos alternativas consideras una mejor práctica? Explica brevemente los motivos.

3. En forma individual, explica qué ventaja adicional ofrece un ViewModel frente a otras formas de mantener el estado al navegar entre pantallas. (Cada integrante del grupo debe incluir su propia respuesta en el informe).

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Android Developers, "Navigation with Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/navigation

[2] Android Developers, "ViewModel overview". [En línea]. Disponible: https://developer.android.com/topic/libraries/architecture/viewmodel

[3] Android Developers, "Navigation bar | Jetpack Compose". [En línea]. Disponible: https://developer.android.com/develop/ui/compose/components/navigation-bar

[4] "Cómo navegar entre pantallas en Jetpack Compose con Room y ViewModel (MVVM)", video. [En línea]. Disponible: https://www.youtube.com/watch?v=1ZuBVRYIr7w

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:	**INSTRUMENTOS:**
Ejercicios propuestos	Rúbrica de evaluación

CRITERIOS DE EVALUACIÓN

- Cumplimiento de los objetivos y ejercicios propuestos en la guía.
- Correcta aplicación de los conceptos y herramientas del tema desarrollado.
- Funcionamiento correcto de lo implementado (compilación/ejecución sin errores).
- Informe (formato estudiante) completo, con evidencias y cuestionario resuelto.
- Entrega dentro del plazo establecido.
- Se permite el uso de Inteligencia Artificial (IA) como fuente de consulta o apoyo para resolver los ejercicios; sin embargo, el desarrollo completo del laboratorio no puede basarse únicamente en respuestas generadas por IA. El estudiante debe evidenciar comprensión propia mediante la justificación razonada de sus respuestas, especialmente en el cuestionario.